

Granular Synchrony

Neil Giridharan
University of California,
Berkeley
Berkeley, CA 94720, USA
+1 (510) 642-6000
neil.giridharan@berkeley
.edu

Ittai Abraham
Intel Labs
Santa Clara, CA 95054, USA
+1 (408) 765-8080
ittai.abraham@intel.com

Natacha Crooks
University of California,
Berkeley
Berkeley, CA 94720, USA
+1 (510) 642-6000
natacha.crooks@berkeley.edu

ABSTRACT

Traditional network timing models—synchrony, partial synchrony, and asynchrony—often oversimplify real-world communication dynamics, making them either overly optimistic or pessimistic. This paper introduces **granular synchrony**, a refined model that integrates a mixture of synchronous, partially synchronous, and asynchronous communication links. This framework captures modern distributed network behaviors more accurately and serves as a unifying approach where existing models become special cases. We explore fault-tolerant consensus mechanisms within granular synchrony and demonstrate that consensus can be achieved despite $f \geq n/2$ crash faults or $f \geq n/3$ Byzantine faults, without requiring full synchrony.

CCS Concepts

• Computer systems organization → Dependable and fault-tolerant systems and networks • Networks → Network reliability • Theory of computation → Distributed algorithms.

Keywords

Fault tolerance; network synchrony; distributed systems; consensus protocols.

1. INTRODUCTION

The timing model of a distributed system plays a crucial role in defining communication reliability and protocol feasibility. Traditionally, networks operate under three timing assumptions: **synchrony**, **asynchrony**, and **partial synchrony**. These models have shaped the theoretical and practical foundations of distributed computing, yet they often fail to capture the diverse and dynamic nature of real-world networks.

- **Synchronous networks** guarantee message delivery within a predefined time bound. However, even a single delay beyond violates the assumption, leading to misclassifications of nodes as faulty.
- **Asynchronous networks**, in contrast, place no constraints on message timing, making deterministic consensus theoretically impossible under the well-known FLP impossibility result.
- **Partial synchrony** attempts to bridge the gap between these two extremes by introducing a Global Stabilization Time (GST), after which communication behaves synchronously. While widely adopted, partial synchrony inherits fault tolerance limits similar to asynchronous models.

While these models provide useful abstractions, they fail to represent the heterogeneous nature of modern distributed systems, which span multiple datacenters, cloud environments, and wide-area networks. Within datacenters, communication is often synchronous, while cross-datacenter interactions introduce timing variations due to congestion and adversarial disruptions.

This paper proposes Granular Synchrony, a refined network timing model that recognizes the diverse communication properties of real-world distributed networks. Instead of applying uniform timing assumptions across all communication links, Granular Synchrony models networks as graphs consisting of mixed link behaviors:

1. **Synchronous links**, where messages arrive within a strict time bound.
2. **Partially synchronous links**, which behave unpredictably before GST but eventually follow timing guarantees.
3. **Asynchronous links**, where delivery occurs without defined timing constraints.

Granular Synchrony serves as a unifying framework, encapsulating synchrony, partial synchrony, and asynchrony as special cases. By leveraging graph-based representations, this model introduces new fault-tolerant properties, expanding feasibility conditions for consensus algorithms.

2. MODEL AND DEFINITIONS

In this work, we consider a distributed system where communication links are **bidirectional** and operate under varying degrees of timing guarantees. We define two key variations of our proposed model:

- **Granular Partial Synchrony**, where each communication link is either synchronous or partially synchronous.
- **Granular Asynchrony**, where links may be synchronous, partially synchronous, or completely asynchronous.

A synchronous link guarantees message delivery within a known delay bound Δ . In contrast, a partially synchronous link provides this guarantee only after some unknown Global Stabilization Time (GST). Asynchronous links do not offer any delivery time bounds but ensure eventual delivery of messages. All links are assumed to be FIFO, reliable, and lossless, delivering every message exactly once.

The system consists of n nodes, among which up to f can be compromised. The adversary is adaptive, capable of corrupting nodes at any point during execution. For crash fault tolerance (CFT), faulty nodes may simply stop responding. In the Byzantine fault-tolerant (BFT) setting, corrupted nodes may behave arbitrarily. In BFT scenarios, we assume the use of digital signatures and public key infrastructure (PKI) to ensure message authenticity and integrity.

We do not assume globally synchronized clocks. Instead, we require only bounded clock drift across

nodes. Time-based actions, such as waiting for messages, operate within known delay ranges.

To model network behavior, we represent the system as an undirected graph $G=(V,E)$, where nodes (vertices) correspond to processes and edges represent communication links. Our protocols are graph-agnostic, meaning they do not depend on explicit knowledge of the network topology or link timing types.

We define the following formal concepts:

- **Definition 1 (Synchronous Path):** A node a is said to have a synchronous path to node b , denoted $a \rightarrow b$, if there exists a sequence of synchronous links connecting a to b , traversing only through correct intermediate nodes.
- **Definition 2 (Set Reachability):** For node sets A and B , we say $A \rightarrow B$ if, for every $b \in B$, there exists some $a \in A$ such that $a \rightarrow b$.
- **Definition 3 (Path Metrics):** The length of a path is the number of links it contains. The synchronous distance between two nodes is defined as the shortest such path. The synchronous diameter $d(G)$ of the graph is the maximum distance between any pair of nodes (a,b) , assuming up to f nodes may be faulty. A similar definition applies to partially synchronous paths, resulting in a diameter $d'(G)$.

These metrics provide insight into message propagation delays across the network and are used to determine timeouts and protocol progress guarantees.

- **Definition 4 (Consensus Conditions):** A distributed consensus protocol must satisfy the following:
 - **Agreement:** No two correct nodes decide on different values.
 - **Termination:** Every correct node eventually decides.
 - **Validity:** If all correct nodes start with the same input value, that value must be chosen as the output.

This formal model lays the groundwork for developing fault-tolerant consensus algorithms that operate effectively in heterogeneous and dynamically changing network conditions.

3. CONSENSUS UNDER GRANULAR PARTIAL SYNCHRONY CFT

The ability to achieve **fault-tolerant consensus** depends on timing guarantees within the underlying network graph. In classical models:

- **Synchronous consensus** tolerates up to $(f < n/2)$ Byzantine faults and $(f < n)$ crash faults.
- **Partial synchrony** limits fault tolerance to $(f < n/3)$ Byzantine faults or $(f < n/2)$ crash faults.
- **Asynchronous models** impose similar thresholds but require **randomization** for feasibility.

Granular Synchrony redefines these fault thresholds, demonstrating that consensus remains achievable even in networks with mixed timing properties. Instead of assuming full synchrony, consensus feasibility relies on the connectivity structure of the network graph.

3.1 Necessity of Connectivity Conditions

For consensus to be achievable in this model, the underlying communication graph must meet specific reachability criteria. In particular, the protocol requires that, for any subset $A \subseteq V$ with at least $n-f$ nodes, there must exist another subset $B \subseteq V$ with at least $f+1$ nodes such that each node in B is reachable from some node in A via a synchronous path. This ensures that messages from a quorum can propagate reliably to a critical subset of nodes, enabling coordination.

This condition prevents partitioning scenarios where groups of correct nodes may be isolated due to the lack of synchronous links, which could lead to divergence in decision values. If the condition is violated, adversaries can exploit the partition to

create indistinguishable executions for different groups of nodes, ultimately violating agreement.

3.2 Protocol

The proposed protocol follows a view-based structure, where each view is coordinated by a designated leader and includes several communication phases:

1. **View Initialization:** Each node begins by sending a status message, containing its current view and locked value, to the leader of the current view.
2. **Proposal Phase:** The leader collects at least $n-f$ status messages and proposes the value with the highest lock observed. This proposed value is then broadcast to all nodes.
3. **Voting Phase:** Upon receiving the proposal, nodes update their lock and send vote messages to all other nodes.
4. **Commit Phase:** If a node gathers $n-f$ matching votes or receives a commit message, it finalizes the decision, broadcasts a commit message, and terminates.
5. **View Change:** If no progress is made within a bounded time, nodes initiate a view change by broadcasting a NewView message. Upon receiving sufficient support for the new view, nodes transition and update their locks accordingly.

This protocol ensures that even if two sets of $n-f$ nodes do not intersect, the condition guarantees that information can flow from one quorum to another via synchronous paths, thus maintaining safety.

3.3 Analysis

Lemma 1: If a node commits a value in view v , no future leader in any view $v' \geq v$ can propose a different value.

Proof Sketch: The committed value must have received $n-f$ votes. Due to the synchronous path requirement, any new leader collecting status from $n-f$ nodes will overlap with the committed quorum and observe the committed value, preserving consistency across views.

Theorem 2 (Agreement): No two correct nodes can commit different values.

Proof: Once a value is committed in view v , Lemma 1 ensures that this is the only value that can be proposed in any subsequent view, ruling out conflicting commitments.

Theorem 3 (Termination): All correct nodes eventually commit a value.

Proof: Given round-robin leader election, a correct leader will eventually be chosen post-GST. From that point on, synchronous paths and timer bounds ensure all messages are delivered on time, enabling progress and eventual commitment within 4Δ time.

Theorem 4 (Validity): If every node starts with the same input value, then that value will be the final decision.

Proof: All nodes begin locked on the same input. Since no other value can be proposed in any view, due to lock propagation and view ranking, the initial value is preserved and ultimately committed.

4. CONSENSUS IN GRANULAR ASYNCHRONY CFT

Under granular asynchrony, CFT consensus on a graph $G = (V, E)$ can be solved deterministically if and only if, (i) the condition in theorem 1 holds and (ii) for all F with $|F| \leq f$, less than $n - f$ nodes are outside the largest connected component of $G' = (V - F, \diamond E)$ where $\diamond E$ is the set of synchronous and partially synchronous edges. In other words, condition (ii) says that if we remove all asynchronous edges and all faulty nodes from G and further remove the largest connected component in the remaining graph, then there are fewer than $n - f$ nodes left.

4.1 Necessity

Deterministic CFT consensus on a graph $G=(V,E)$ is achievable under granular asynchrony if and only if:

1. The condition established in Theorem 1 (synchronous reachability from $n-f$ to $f+1$ nodes) holds.

2. For every subset $F \subseteq V$ where $|F| \leq f$, fewer than $n-f$ nodes lie outside the largest connected component of the reduced graph $G'=(V-F, \diamond E)$, where $\diamond E$ denotes only the synchronous and partially synchronous edges.

While the first condition is already proven necessary, we now justify the second. Suppose the graph violates condition (ii) but is still claimed to solve consensus via some deterministic algorithm A . Then there must exist a fault set F of size $\leq f$, such that removing F and all asynchronous edges results in a graph where $n-f$ or more nodes are not part of the largest remaining component.

We now simulate this scenario by constructing an external system with q isolated components, each corresponding to a fragment of the original graph. If each component is simulated independently and communicates only through asynchronous channels, a consensus protocol functioning under such conditions would violate the FLP impossibility result, which states that no deterministic algorithm can guarantee consensus in purely asynchronous systems with even a single crash fault. Therefore, the condition must be necessary.

4.2 Protocol Adaptation for Granular Asynchrony

To handle the more general timing assumptions, we adapt the previous leader-based CFT protocol from granular partial synchrony to work under granular asynchrony. The changes primarily affect the **status** and **view-change** phases, while the rest of the consensus flow—proposal, vote, and commit—remains intact.

Upon entering a new view v , each node broadcasts a $\langle \text{Status}, v, \text{lock} \rangle$ message. When a node receives $n-f$ such messages, it forwards this collection to all nodes and starts a **proposal timer** of duration $3d'\Delta/3d'\Delta$, where d' is the partially synchronous diameter of the graph.

If a message is received in time, the node locks onto the proposed value, forwards the proposal, and votes accordingly. If no valid proposal is received before the timer expires, the node suspects a faulty leader.

If a node does not receive a proposal in time, it sends a message to all nodes. Upon receiving $n-f$ such messages for the same view, it initiates a and transitions into the next view. This ensures progress despite asynchronous delays or leader failures.

Importantly, we require $n-f$ nodes to participate in initiating a view change. This threshold, in combination with the topological condition ensures that a stable, correct leader in the largest connected component will eventually maintain control long enough to finalize consensus.

4.3 Protocol Adaptation

To handle the more general timing assumptions, we adapt the previous leader-based CFT protocol from granular partial synchrony to work under granular asynchrony. The changes primarily affect the **status** and **view-change** phases, while the rest of the consensus flow—proposal, vote, and commit—remains intact.

Upon entering a new view v , each node broadcasts a message. When a node receives $n-f$ such messages, it forwards this collection to all nodes and starts a **proposal timer** of duration $3d'\Delta$, where d' is the partially synchronous diameter of the graph.

If a message is received in time, the node locks onto the proposed value, forwards the proposal, and votes accordingly. If no valid proposal is received before the timer expires, the node suspects a faulty leader.

If a node does not receive a proposal in time, it sends a message to all nodes. Upon receiving $n-f$ such messages for the same view, it initiates and transitions into the next view. This ensures progress despite asynchronous delays or leader failures.

Importantly, we require $n-f$ nodes to participate in initiating a view change. This threshold, in combination with the topological condition in Theorem 5, ensures that a stable, correct leader in the largest connected component will eventually maintain control long enough to finalize consensus.

4.4 Termination Guarantee

The safety and validity properties follow directly from the quorum-based mechanics and lock propagation mechanisms, as previously proven.

The focus now shifts to demonstrating **eventual termination** under the proposed conditions.

Lemma 2: If no correct node ever terminates, then all correct nodes must continuously move to higher views.

Proof Summary: Suppose a correct node n_{1n_1n1} stops progressing and remains in view v . Eventually, another correct node will enter a higher view and broadcast a message, which will reach n_{1n_1n1} , forcing it to transition as well. This contradiction implies all correct nodes must continue advancing views until consensus is achieved.

Theorem 6 (Termination): All correct nodes will eventually decide.

Proof Sketch: Assume no correct node ever decides. Eventually, a correct node within the largest connected component of the fault-free, non-asynchronous graph G' becomes leader in some view v after stabilization (GST). Let t be the time the first node in C enters view v , where $t > GST + 2d$.

Since the faulty nodes are all in F , and the number of nodes outside $C \cup F$ is less than $n-f$, not enough view change messages can be generated to force a leader replacement. As a result, the leader in C receives $n-f$ messages and sends a valid message within bounded time. The message is forwarded, votes are cast, and all correct nodes in C receive enough messages to commit.

Eventually, the commit message disseminates, and all correct nodes terminate. Thus, a contradiction arises if we assume any node fails to decide, confirming eventual consensus.

5. BYZANTINE FAULT-TOLERANT CONSENSUS IN GRANULAR PARTIAL SYNCHRONY

Under granular partial synchrony, BFT consensus with $n \geq 2f + 1$ on a graph G is solvable if and only if, for any set F of at most f faulty nodes, $\forall A \subseteq V - F$ with $|A| \geq n - 2f$, $\exists B \subseteq V - F$ with $|B| \geq f + 1$

such that $A \rightarrow B$. In words, the condition is that any honest set A of size at least $n - 2f$ has a potentially larger honest set B of size at least $f + 1$, such that for any node $b \in B$ there exists $a \in A$ and a synchronous path from a to b . Intuitively, if a message arrives at all of A , then it will also arrive at all of B after some delay.

5.1 Necessary

To ensure the safety and liveness of Byzantine fault-tolerant (BFT) consensus in a network where links may differ in synchrony, it is essential to avoid scenarios where correct nodes become isolated or form disjoint groups unable to reach each other in time. A central requirement is that even in the presence of up to f Byzantine nodes, a group of $n - 2f$ correct nodes must be able to reliably reach a set of at least $f + 1$ other correct nodes through synchronous paths. This avoids the risk of adversarial influence creating inconsistent decisions across partitions of the system.

This necessity becomes apparent when one imagines the classic "split-brain" problem: two large groups of correct nodes, each manipulated to believe they have achieved consensus, while being isolated from each other due to delays or faulty nodes. Without guaranteed synchronous reachability between sufficient subsets of correct nodes, consensus cannot be safely maintained.

5.2 Consensus Protocol Design

To operate within this setting, we design a BFT consensus protocol that extends the standard partially synchronous leader-based model. The key adaptation is the promotion of small groups of reliable nodes through the use of a specialized quorum condition leveraging graph connectivity.

Each consensus round (or "view") begins with a **status collection phase**, followed by leader proposal, locking, and commitment. Below are the summarized steps:

- **Status Phase:** Every node sends a status message to the current leader at the start of each view.
- **Leader Proposal:** Upon receiving $n - f$ status messages, the leader selects the highest ranked lock (if any). If no lock is

provided, it can propose its own input. It then sends a signed proposal to all nodes.

- **Equivocation Detection:** Nodes forward valid proposals and start a timer. If multiple conflicting proposals are received, nodes flag equivocation and initiate a view change.
- **Locking:** When $n - f$ votes for the same value are received, a lock is created and propagated. This ensures a single proposal per view is enforced.
- **Commitment:** Upon obtaining $n - f$ second-round votes for a value, the node commits and notifies all other nodes.

5.3 Safety and Termination Guarantees

We now verify the correctness of the protocol with respect to consensus properties.

- **Agreement:** We ensure that no two correct nodes ever commit different values. This follows from the quorum intersection principle—any two sets of $n - f$ nodes must overlap in at least one correct node. This node guarantees the consistency of the highest locked value passed to future views.
- **Termination:** Progress is ensured through the rotating leader mechanism. Eventually, a correct leader will be chosen post-global stabilization time (GST), at which point all correct nodes will converge on the same value and complete the protocol.

5.4 Unanimity Validity

While the above protocol achieves external validity, it can be further strengthened to unanimity validity, where all correct nodes having the same input must result in that value being decided.

To achieve this, nodes perform an initial phase before view 1, where they exchange and echo input values. If all correct nodes hold the same input, they will form a unanimous initial lock on that value. From this point, the locking and proposal mechanism naturally upholds the unanimity condition through all views.

6. CONCLUSION

Granular Synchrony redefines network timing models, offering a more precise and flexible framework for distributed computing. It provides stronger fault tolerance guarantees, bridging the gap between classical timing models and modern heterogeneous networks. By incorporating mixed link behaviors into consensus protocols, Granular Synchrony expands feasibility conditions, making distributed systems more robust against uncertainty. Future research will focus on applying this model to blockchain networks, large-scale cloud environments, and decentralized infrastructures.

7. ACKNOWLEDGMENTS

This work was initiated while Ittai Abraham, Neil Girdharan, and Ling Ren were affiliated with VMware Research. The research was partially supported by the National Science Foundation under award number.

8. REFERENCES

[1] Ittai Abraham, Dahlia Malkhi, Kartik Nayak, Ling Ren, and Maofan Yin. Sync hotstuff: Simple and practical synchronous state machine replication. In 2020 IEEE Symposium on Security and Privacy (SP), pages 106–118, 2020. doi:10.1109/SP40000.2020.00044.

[2] Ittai Abraham, Kartik Nayak, Ling Ren, and Zhuolun Xiang. Good-case latency of byzantine broadcast: A complete categorization. In Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing, PODC’21, page 331–341, New York, NY, USA, 2021. Association for Computing Machinery. doi:10.1145/3465084.3467899.

[3] Saksham Agarwal, Qizhe Cai, Rachit Agarwal, David Shmoys, and Amin Vahdat. Harmony: A congestion-free datacenter architecture. In 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24), pages 329–343, Santa Clara, CA, April 2024. USENIX Association. URL: <https://www.usenix.org/conference/nsdi24/presentation/agarwal-saksham>.

[4] Marcos K. Aguilera, Carole Delporte-Gallet, Hugues Fauconnier, and Sam Toueg. Communication efficient leader election and consensus with limited link synchrony. In Proceedings of the Twenty-Third Annual ACM Symposium on Principles of Distributed Computing, PODC ’04, page 328–337, New York, NY, USA, 2004. Association for Computing Machinery. doi:10.1145/1011767.1011816.

[5] M.K. Aguilera, C. Delporte-Gallet, H. Fauconnier, and S. Toueg. Consensus with byzantine failures and little system synchrony. In International Conference on Dependable Systems and Networks (DSN’06), pages 147–155, 2006. doi:10.1109/DSN.2006.22.

[6] Ailidani Ailijiang, Aleksey Charapko, Murat Demirbas, and Tevfik Kosar. Wpaxos: Wide area network flexible consensus. IEEE Trans. Parallel Distrib. Syst., 31(1):211–223, jan 2020. doi:10.1109/TPDS.2019.2929793.

[7] Olivier Baldellon, Achour Mostéfaoui, and Michel Raynal. A necessary and sufficient synchrony condition for solving byzantine consensus in symmetric networks. In International Conference on Distributed Computing and Networking, pages 215–226. Springer, 2011.

[8] Zohir Bouzid, Achour Mostfaoui, and Michel Raynal. Minimal synchrony for byzantine consensus. In Proceedings of the 2015 ACM Symposium on Principles of Distributed Computing, pages 461–470, 2015.

[9] Gabriel Bracha. Asynchronous byzantine agreement protocols. Information and Computation, 75(2):130–143, 1987.